35.2 控制台 UI 架构

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~140k tokens，输出 ~8k tokens（本节）

上一节介绍了控制台 UI 的技术选型（Lit + Vite + Legacy Decorators）。本节从架构层面展开：Gateway 如何将这个 SPA 嵌入自身的 HTTP 服务、前端源码如何组织、以及浏览器端的 WebSocket 客户端如何与 Gateway 通信。

35.2.1 Gateway 内嵌 UI 服务

控制台 UI 的构建产物（HTML、JS、CSS、SVG）打包后存放在 `dist/control-ui/` 目录下。Gateway 进程启动时，不需要额外的 Nginx 或 CDN——它自己就是 UI 的 HTTP 服务器。核心逻辑在 `src/gateway/control-ui.ts` 。

静态文件服务与 SPA 回退

`handleControlUiHttpRequest` 函数处理所有 UI 相关的 HTTP 请求，其流程如下：

```
// src/gateway/control-ui.ts（简化）
export function handleControlUiHttpRequest(
  req: IncomingMessage,
  res: ServerResponse,
  opts?: ControlUiRequestOptions,
): boolean {
  // 1. 仅接受 GET / HEAD
  if (req.method !== "GET" && req.method !== "HEAD") {
    res.statusCode = 405;
    res.end("Method Not Allowed");
    return true;
  }

  // 2. 解析 basePath（支持反向代理挂载到子路径）
  const basePath = normalizeControlUiBasePath(opts?.basePath);

  // 3. 定位静态资源根目录
  const root = resolveControlUiRootSync({ ... });
  if (!root) {
    res.statusCode = 503;
    res.end("Control UI assets not found...");
    return true;
  }

  // 4. 尝试匹配文件；若文件存在则直接返回
  const filePath = path.join(root, fileRel);
  if (fs.existsSync(filePath) && fs.statSync(filePath).isFile()) {
    serveFile(res, filePath);
    return true;
  }

  // 5. SPA 回退：所有未匹配路径返回 index.html
  const indexPath = path.join(root, "index.html");
  serveIndexHtml(res, indexPath, { basePath, config, agentId });
  return true;
}
```

这是经典的 **SPA 回退模式**：前端使用 History API 进行客户端路由（如 `/chat` 、 `/agents` ），但这些路径在服务端并没有对应的物理文件。Gateway 将所有未匹配的 GET 请求重定向到 `index.html` ，由前端 JavaScript 根据 URL 决定显示哪个页面。

> **衍生解释 — SPA 回退（History API Fallback）**

> 单页应用（SPA）使用浏览器的 History API（`pushState` / `popstate`）实现无刷新页面切换。当用户直接访问 `/agents` 或刷新页面时，浏览器会向服务器请求 `/agents` 路径。如果服务器没有对应文件，就需要"回退"到 `index.html`，让前端路由接管。这就是为什么 Nginx 配置 SPA 时常见 `try_files $uri /index.html;` 规则。

安全头注入

每个响应都附带安全头：

```
function applyControlUiSecurityHeaders(res: ServerResponse) {
  res.setHeader("X-Frame-Options", "DENY");
  res.setHeader("Content-Security-Policy", "frame-ancestors 'none'");
  res.setHeader("X-Content-Type-Options", "nosniff");
}
```

安全头	值	作用
`X-Frame-Options`	`DENY`	禁止被嵌入到 `<iframe>` 中，防止点击劫持
`Content-Security-Policy`	`frame-ancestors 'none'`	CSP 级别的 iframe 嵌入禁止
`X-Content-Type-Options`	`nosniff`	阻止浏览器对 MIME 类型的嗅探猜测

配置注入：index.html 动态写入

Gateway 在返回 `index.html` 前会动态注入一段 `<script>` 标签，将运行时配置传递给前端：

```
function injectControlUiConfig(html: string, opts: ControlUiInjectionOpts): string {
  const script =
    `<script>` +
    `window.__OPENCLAW_CONTROL_UI_BASE_PATH__=${JSON.stringify(basePath)};` +
    `window.__OPENCLAW_ASSISTANT_NAME__=${JSON.stringify(assistantName)};` +
    `window.__OPENCLAW_ASSISTANT_AVATAR__=${JSON.stringify(assistantAvatar)};` +
    `</script>`;
  // 注入到 </head> 之前
  const headClose = html.indexOf("</head>");
  return `${html.slice(0, headClose)}${script}${html.slice(headClose)}`;
}
```

这种“HTML 注入”模式的优势是：构建产物是静态的，但每个 Gateway 实例可以有不同的助手名称和头像，无需重新构建。前端通过 `window.__OPENCLAW_CONTROL_UI_BASE_PATH__` 等全局变量读取这些值。

头像端点

除了静态文件服务，Gateway 还提供专用的头像端点 `/_avatar/{agentId}` 。 `handleControlUiAvatarRequest` 支持两种模式：

1. **图片直出**（默认）：如果头像是本地文件，直接返回二进制内容
2. **元数据查询**（ `?meta=1` ）：返回 JSON `{ avatarUrl }` 供前端决策——头像可能是本地文件、远程 URL 或 data URI

35.2.2 UI 源码结构

前端代码位于 `ui/src/ui/` 目录下，约 50 个文件，采用“单组件 + 模块分解”的架构。

整体文件组织

```
ui/src/ui/
├── app.ts                    # OpenClawApp 主组件（LitElement 子类）
├── app-view-state.ts         # AppViewState 类型定义（渲染契约）
├── app-render.ts             # renderApp() 入口 → 分发到各视图
├── app-render.helpers.ts     # 渲染辅助函数（Tab 渲染、主题切换）
├── app-gateway.ts            # Gateway WebSocket 连接与事件分发
├── app-lifecycle.ts           # Lit 生命周期钩子 Agent
├── app-settings.ts           # 设置持久化、URL 参数同步、Tab 切换
├── app-chat.ts               # 聊天发送/中止/队列
├── app-channels.ts           # WhatsApp/Nostr 等通道管理
├── app-scroll.ts             # 聊天/日志自动滚动
├── app-polling.ts            # Nodes/Logs/Debug 轮询
├── app-tool-stream.ts        # 工具调用流式显示
├── app-events.ts             # 事件日志类型
├── app-defaults.ts           # 默认值常量
├── navigation.ts             # 12 Tab 定义与路由
├── gateway.ts                # GatewayBrowserClient 类
├── storage.ts                # localStorage 持久化
├── types.ts                  # 共享类型（~775 行）
├── ui-types.ts               # UI 特定类型
├── views/                    # 各 Tab 的视图渲染函数
│   ├── chat.ts, agents.ts, channels.ts, config.ts, ...
│   └── （共 15+ 视图文件）
├── controllers/              # 各功能的数据加载/操作函数
│   ├── chat.ts, agents.ts, config.ts, sessions.ts, ...
│   └── （共 23 个控制器文件）
├── chat/                     # 聊天消息渲染辅助
│   ├── grouped-render.ts, message-normalizer.ts, ...
│   └── （共 10 个文件）
└── components/               # 可复用 UI 组件
    └── resizable-divider.ts
```

单组件架构

整个 UI 只有一个 Lit 组件 `OpenClawApp` （`<openclaw-app>` 标签），包含 **100+ 个 `@state()` 响应式字段**。这不是“组件化”的教科书做法，但对于管理面板类应用来说，有其合理性：

1. **状态集中管理**：所有状态在一处，不需要 Context/Store 等状态管理方案
2. **避免组件间通信开销**：父子组件、兄弟组件之间的 props 传递和事件冒泡在只有一个组件时完全消除

3. **降低 Lit 特有的序列化成本**：Custom Elements 的 attribute 传值只支持字符串，复杂对象需要 property 传递

代价是这个组件非常庞大。OpenClaw 的应对策略是 **模块分解**——将逻辑拆到 20+ 个独立文件中：

```
// ui/src/ui/app.ts（简化结构）
@customElement("openclaw-app")
export class OpenClawApp extends LitElement {
  // ── 状态声明（100+字段）──
  @state() settings: UiSettings = loadSettings();
  @state() connected = false;
  @state() tab: Tab = "chat";
  @state() chatMessages: unknown[] = [];
  // ...

  // ── 不使用 Shadow DOM ──
  createRenderRoot() { return this; }

  // ── 生命周期委托 ──
  connectedCallback()   { super.connectedCallback(); handleConnected(this); }
  firstUpdated()        { handleFirstUpdated(this); }
  disconnectedCallback(){ handleDisconnected(this); super.disconnectedCallback(); }
  updated(changed)      { handleUpdated(this, changed); }

  // ── 方法委托到各模块 ──
  connect()             { connectGateway(this); }
  applySettings(next)   { applySettingsInternal(this, next); }
  setTab(next)          { setTabInternal(this, next); }
  async handleSendChat(){ await handleSendChatInternal(this); }

  // ── 渲染委托 ──
  render() { return renderApp(this); }
}
```

注意 `createRenderRoot() { return this; }` 这行——它跳过了 Shadow DOM，让组件直接渲染到 Light DOM 中。这意味着全局 CSS（`styles.css`）可以直接作用于组件内部元素，无需 CSS Modules 或 `::part()` 伪元素。

> **衍生解释 — Light DOM vs Shadow DOM**
>
> Shadow DOM 是 Web Components 的核心特性之一，它为组件创建一个隔离的 DOM 子树，外部 CSS 无法穿透进去。这对于通用组件库（如 Material Design 组件）很有价值，但对于应用级组件

> 来说，样式隔离反而是障碍——你需要额外的机制来共享主题变量。OpenClaw 选择 Light DOM 是务实之举。

渲染模型：AppViewState 契约

`OpenClawApp` 类和 `renderApp()` 函数之间通过 `AppViewState` 类型建立契约：

```
// ui/src/ui/app-view-state.ts（节选）
export type AppViewState = {
  // 数据字段
  settings: UiSettings;
  connected: boolean;
  tab: Tab;
  chatMessages: unknown[];
  sessionKey: string;
  // ...

  // 行为方法
  connect: () => void;
  setTab: (tab: Tab) => void;
  applySettings: (next: UiSettings) => void;
  handleSendChat: (msg?: string) => Promise<void>;
  resetToolStream: () => void;
  scrollToBottom: () => void;
  // ... 共 40+ 个方法
};
```

`AppViewState` 既包含数据字段，也包含行为方法。 `renderApp()` 接收这个类型而非 `OpenClawApp` 类本身——这意味着渲染函数只依赖接口，而非具体实现，便于测试和重构。

视图分发

`renderApp()` 函数根据当前 Tab 选择性渲染对应视图：

```
// ui/src/ui/app-render.ts（简化）
export function renderApp(state: AppViewState) {
  return html`
    <div class="shell ${state.tab === 'chat' ? 'shell--chat' : ''}">
      <header class="topbar">...</header>
      <aside class="nav">
        ${TAB_GROUPS.map(group => html`
          <div class="nav-group">
            ${group.tabs.map(tab => renderTab(state, tab))}
          </div>
        `)}
      </aside>
      <main class="content">
        ${state.tab === "chat"     ? renderChat({...})     : nothing}
        ${state.tab === "agents"   ? renderAgents({...})   : nothing}
        ${state.tab === "overview" ? renderOverview({...}) : nothing}
        ${state.tab === "config"   ? renderConfig({...})   : nothing}
        <!-- ... 共 12 个 Tab 视图 -->
      </main>
    </div>
  `;
}
```

这里使用了 Lit 的 `nothing` 特殊值——当条件不满足时，Lit 不会为该位置生成任何 DOM 节点，实现了懒渲染。

导航系统

12 个 Tab 分为 4 组：

组	Tab	说明
Chat	`chat`	WebChat 聊天界面
Control	`overview` , `channels` , `instances` , `sessions` , `usage` , `cron`	运维管理
Agent	`agents` , `skills` , `nodes`	Agent 配置
Settings	`config` , `debug` , `logs`	系统设置

路由机制基于浏览器 History API，每个 Tab 对应一个路径（如 `/agents` ）：

```ts
// ui/src/ui/navigation.ts
const TAB_PATHS: Record<Tab, string> = {
  chat: "/chat", overview: "/overview", channels: "/channels",
  instances: "/instances", sessions: "/sessions", usage: "/usage",
  cron: "/cron", agents: "/agents", skills: "/skills",
  nodes: "/nodes", config: "/config", debug: "/debug", logs: "/logs",
};

export function tabFromPath(pathname: string, basePath = ""): Tab | null {
  // 剥离 basePath 前缀
  let path = pathname;
  if (basePath && path.startsWith(`${basePath}/`)) {
    path = path.slice(basePath.length);
  }
  // 根路径默认到 chat
  if (normalizePath(path) === "/") return "chat";
  return PATH_TO_TAB.get(normalizePath(path)) ?? null;
}
```

`inferBasePathFromPathname` 函数支持自动推断 basePath——当 Gateway 被反向代理挂载到 `/admin/ui/` 等子路径时，前端能自动感知并正确生成链接。

控制器层

`controllers/` 目录包含 23 个文件，每个文件封装一个功能域的数据操作。控制器不是类，而是纯函数，接收 `OpenClawApp` 实例（或其类型投影）作为参数：

```
controllers/
├── chat.ts           # 聊天历史加载 + 聊天事件处理
├── agents.ts         # Agent 列表加载
├── config.ts         # 配置读写 + Schema 加载
├── sessions.ts       # 会话列表 + CRUD
├── channels.ts       # 通道状态查询
├── skills.ts         # 技能管理
├── nodes.ts          # 节点列表
├── devices.ts        # 设备配对管理
├── cron.ts           # 定时任务 CRUD
├── usage.ts          # 用量统计
├── logs.ts           # 日志拉取
├── debug.ts          # Debug RPC 调用
├── presence.ts       # 在线实例
├── exec-approval.ts # 执行审批（单条）
├── exec-approvals.ts# 执行审批（批量策略）
├── assistant-identity.ts  # 助手身份
├── agent-identity.ts      # 助手身份
├── agent-files.ts         # Agent 工作区文件
├── agent-skills.ts        # Agent 技能报告
└── config/                # 配置编辑器子模块
```

所有控制器通过 `host.client.request(method, params)` 与 Gateway WebSocket 通信。例如加载 Agent 列表：

```
// ui/src/ui/controllers/agents.ts（概念性示例）
export async function loadAgents(host: OpenClawApp) {
  host.agentsLoading = true;
  host.agentsError = null;
  try {
    const result = await host.client!.request<AgentsListResult>("agents.list");
    host.agentsList = result;
  } catch (err) {
    host.agentsError = String(err);
  } finally {
    host.agentsLoading = false;
  }
}
```

这种模式的特点是：**控制器直接修改宿主组件的 `@state()` 字段**。由于 `@state()` 字段的变更会自动触发 Lit 的重渲染，控制器无需手动通知 UI 更新。

35.2.3 Gateway WebSocket 客户端

前端与 Gateway 的所有实时通信都通过一个 WebSocket 连接完成，封装在 `GatewayBrowserClient` 类中。

连接协议

连接建立的流程是：

```
浏览器                                  Gateway
  │                                        │
  ├─── WebSocket open ──────────────────→│
  │                                        │
  │←─── event: connect.challenge ────────┤  (携带 nonce)
  │                                        │
  ├─── req: connect (附带签名) ──────────→│
  │                                        │
  │←─── res: hello-ok (附带 snapshot)──┤
  │                                        │
  │←─── event: chat/agent/presence ──────┤  (持续推送)
  ├─── req: agents.list ────────────────→│
  │←─── res: { agents: [...] } ──────────┤
```

WebSocket 打开后，Gateway 先发送 `connect.challenge` 事件携带一个 nonce（随机数），客户端用它进行签名认证后发送 `connect` 请求。认证通过后 Gateway 返回 `hello-ok` 。

帧格式

通信使用三种 JSON 帧：

```typescript
// 事件帧（Gateway → 浏览器，推送）
type GatewayEventFrame = {
  type: "event";
  event: string;       // "chat" | "agent" | "presence" | "cron" | ...
  payload?: unknown;
  seq?: number;        // 单调递增序列号
};

// 请求帧（浏览器 → Gateway，RPC）
// { type: "req", id: "uuid", method: "agents.list", params: {...} }

// 响应帧（Gateway → 浏览器，RPC 回复）
type GatewayResponseFrame = {
  type: "res";
  id: string;          // 匹配请求的 UUID
  ok: boolean;
  payload?: unknown;
  error?: { code: string; message: string };
};
```

事件帧是服务端主动推送，请求/响应帧是客户端发起的 RPC 调用。序列号 `seq` 用于检测事件是否丢失——如果收到的 seq 不是上次 +1，客户端会触发 `onGap` 回调提示用户。

设备身份认证

`GatewayBrowserClient` 支持 Ed25519 设备身份认证。首次连接时，浏览器使用 Web Crypto API 生成密鑰对，存入 IndexedDB。后续连接时用私鑰对 `{deviceId, clientId, nonce, signedAtMs, ...}` 进行签名：

```ts
// ui/src/ui/gateway.ts（简化）
private async sendConnect() {
  const isSecureContext = typeof crypto !== "undefined" && !!crypto.subtle;

  if (isSecureContext) {
    // 加载或创建 Ed25519 密钥对
    const deviceIdentity = await loadOrCreateDeviceIdentity();
    // 尝试使用已存储的设备 token
    const storedToken = loadDeviceAuthToken({ deviceId, role })?.token;
    authToken = storedToken ?? this.opts.token;

    // 对连接参数签名
    const payload = buildDeviceAuthPayload({
      deviceId, clientId, clientMode, role, scopes, signedAtMs, token, nonce,
    });
    const signature = await signDevicePayload(privateKey, payload);
    device = { id: deviceId, publicKey, signature, signedAt, nonce };
  }

  const hello = await this.request<GatewayHelloOk>("connect", {
    minProtocol: 3, maxProtocol: 3,
    client: { id: "openclaw-control-ui", mode: "webchat", ... },
    role: "operator", scopes: ["operator.admin", ...],
    device, auth: { token: authToken, password },
  });

  // 保存 Gateway 颁发的设备 token 供下次使用
  if (hello.auth?.deviceToken) {
    storeDeviceAuthToken({ deviceId, role, token: hello.auth.deviceToken });
  }
}
```

> **衍生解释 — Ed25519**
>
> Ed25519 是一种椭圆曲线数字签名算法，属于 Edwards-curve Digital Signature Algorithm (EdDSA) 家族。它的特点是：密钥短（公钥 32 字节）、签名快、验证快，且没有像 ECDSA 那样的随机数陷阱（确定性签名）。Web Crypto API 在安全上下文（HTTPS 或 localhost）中提供了对 Ed25519 的原生支持。OpenClaw 使用 `@noble/ed25519` 作为兼容层。

在非安全上下文（如 HTTP 明文访问）中， `crypto.subtle` 不可用，客户端会退回到纯 token 认证模式。

自动重连与退避

连接断开后，客户端自动按指数退避策略重连：

```
private backoffMs = 800;  // 初始延迟

private scheduleReconnect() {
  if (this.closed) return;
  const delay = this.backoffMs;
  this.backoffMs = Math.min(this.backoffMs * 1.7, 15_000);  // 最大 15 秒
  window.setTimeout(() => this.connect(), delay);
}
```

重连序列：800ms → 1360ms → 2312ms → 3930ms → ... → 15000ms（封顶）。连接成功后 `backoffMs` 重置为 800ms。特殊情况：WebSocket 关闭码 1012（Service Restart）不会显示错误信息——这通常发生在用户通过 UI 保存配置后 Gateway 重启的场景。

事件分发

连接建立后，Gateway 持续推送事件。 `handleGatewayEvent` 函数根据事件类型分发到不同处理器：

```
// ui/src/ui/app-gateway.ts（简化）
function handleGatewayEventUnsafe(host: GatewayHost, evt: GatewayEventFrame) {
  // 事件日志（保留最近 250 条）
  host.eventLogBuffer = [
    { ts: Date.now(), event: evt.event, payload: evt.payload },
    ...host.eventLogBuffer,
  ].slice(0, 250);

  switch (evt.event) {
    case "agent":
      handleAgentEvent(host, evt.payload);       // 工具流式输出
      break;
    case "chat":
      handleChatEvent(host, evt.payload);         // 聊天消息更新
      break;
    case "presence":
      host.presenceEntries = evt.payload.presence; // 在线实例列表
      break;
    case "cron":
      loadCron(host);                             // 刷新定时任务
      break;
    case "device.pair.requested":
    case "device.pair.resolved":
      loadDevices(host);                          // 刷新设备列表
      break;
    case "exec.approval.requested":
      // 添加到审批队列，设置过期定时器
      const entry = parseExecApprovalRequested(evt.payload);
      host.execApprovalQueue = addExecApproval(host.execApprovalQueue, entry);
      setTimeout(() => removeExecApproval(host.execApprovalQueue, entry.id),
                 entry.expiresAtMs - Date.now() + 500);
      break;
  }
}
```

特别值得注意的是执行审批事件的处理：当 Agent 请求执行某个需要批准的操作时，Gateway 推送 `exec.approval.requested` 事件，UI 将其加入队列并显示审批弹窗。审批条目有过期时间，到期后自动从队列中移除。

初始快照

`hello-ok` 响应中可能携带 `snapshot` 字段，包含在线实例列表和健康状态：

```
export function applySnapshot(host: GatewayHost, hello: GatewayHelloOk) {
  const snapshot = hello.snapshot as {
    presence?: PresenceEntry[];
    health?: HealthSnapshot;
    sessionDefaults?: SessionDefaultsSnapshot;
  } | undefined;

  if (snapshot?.presence) host.presenceEntries = snapshot.presence;
  if (snapshot?.health)   host.debugHealth = snapshot.health;
  if (snapshot?.sessionDefaults) applySessionDefaults(host, snapshot.sessionDefaults);
}
```

这避免了连接后立即发起多个 RPC 请求来获取初始数据——Gateway 在握手时就将关键状态一次性推送过来。

生命周期编排

`app-lifecycle.ts` 将 Lit 组件的生命周期钉子组织成清晰的启动/更新/销毁流程：

Lit 钩子	操作
`connectedCallback`	推断 basePath → 从 URL 读取设置 → 同步 Tab → 同步主题 → 监听 `popstate` → 连接 Gateway → 启动 Nodes 轮询
`firstUpdated`	监听 topbar 元素尺寸（ResizeObserver）
`updated(changed)`	聊天内容变化时自动滚动到底部；日志内容变化时跟踪滚动
`disconnectedCallback`	清理事件监听、停止轮询、断开 Gateway

`handleUpdated` 中的自动滚动逻辑体现了细致的 UX 考量：只有在用户没有手动上滚时才自动滚动到底部（`chatHasAutoScrolled` 标记），切换到 Chat Tab 时强制滚动。

本节小结

1. **Gateway 自身就是 UI 的 HTTP 服务器**，通过静态文件服务 + SPA 回退 + HTML 配置注入的方式提供控制台界面，无需额外的前端部署。
2. **单组件 + 模块分解**是核心架构：一个 100+ 字段的 `OpenClawApp` 组件承载全部状态，但逻辑分散到 `app-*.ts`（行为）、`views/`（渲染）、`controllers/`（数据）三层。

3. **`AppViewState` 类型**定义了组件到渲染函数的契约，包含数据字段和行为方法，实现了渲染逻辑与组件实现的解耦。
4. **GatewayBrowserClient** 封装了 WebSocket 通信，支持三种帧格式（事件/请求/响应）、Ed25519 设备身份认证、指数退避重连、序列号间隙检测。
5. **事件分发器**根据事件类型路由到对应处理器，支持聊天、工具流、在线状态、定时任务、设备配对、执行审批等事件类型。
6. **初始快照机制**在握手时就推送关键状态，减少连接后的请求往返。